

Beyond Instruction Optimization: Evolutionary Exemplar Selection for Automatic Prompt Optimization

Thanh Vinh Hoang^{1*}, Hoang Viet Nguyen^{1*}, Quang Tung Luu^{1*}
and Thi Thanh Van Tran^{1*}

^{1*}University of Engineering and Technology, Vietnam National
University Hanoi.

*Corresponding author(s). E-mail(s): 23020174@vnu.edu.vn;
23020170@vnu.edu.vn; 23020150@vnu.edu.vn; 23020168@vnu.edu.vn;

Abstract

Large Language Models (LLMs) have demonstrated impressive capabilities to solve a wide range of tasks; however, their performance relies heavily on prompt quality. Designing such well-crafted prompts requires manual writing, onerous trial-and-error and domain knowledge, inaccessible to non-experts. Recent work on automatic prompt optimization (APO) techniques have eased the pain of manual prompt writing. Nevertheless, existing approaches, mainly focus on optimizing instructions and overlook the impact of optimizing exemplars. In this work, we extend the original APO framework by introducing an explicit exemplar optimization stage. Experiments on multiple classification benchmarks demonstrate consistent improvements over frameworks that focus only on instruction optimization, achieving higher average accuracy with minimal additional computation overhead. These findings underscore the significance of exemplar optimization in automatic prompt engineering.

Keywords: APO, Prompt, LLM, GA, ProTeGi

1 Introduction

Recent advances in large language models (LLMs) have demonstrated the impressive capabilities of LLMs in handling various NLP tasks [1]. These LLMs use user’s prompts

input to follow instruction, and thus the performance is affected by those prompts quality. Designing prompts manually is tedious, requires extensive trial-and-error and often demands domain expertise. Consequently, a body of work has investigated automatic prompt optimization (APO) - designing prompt automatically without human intervention.

To adapt to the setting where Large Language Models are mainly accessed through APIs, many APO techniques have been proposed using LLMs as optimizers due to their reasoning and text understanding capabilities [2]. Early approaches explore prompt improvement through search-based strategies, such as Monte Carlo sampling over the space of prompts.[3], which progressively explore the prompt space based on evaluation metrics. More recent methods incorporate gradient-inspired optimization techniques, notably ProTeGi[4], which refines instruction prompts using textual gradients generated from error cases, combined with beam search to avoid the problem of finding local optima. In parallel, evolutionary approaches, such as genetic and population-based prompt optimization methods (e.g., EvoPrompt[5]), have demonstrated the effectiveness of mutation and selection mechanisms to discover high-performing prompts. These techniques possess two significant advantages:

- **Black-box Compatibility:** They do not require access to the model’s internal parameters, gradients, or states.
- **Interpretability:** They maintain human-readable prompts throughout the optimization process, facilitating easier debugging and analysis.

Motivated by these observations, we propose an extension to the ProTeGi [4] framework that explicitly integrates an exemplar optimization step into the automatic prompt optimization process. Our method formulates the exemplar selection as a genetic search problem, maximizes downstream task performance while penalizing excessive prompt length and encouraging exemplar diversity for more generalizable prompt.

Our contributions are as follows:

1. We introduce an exemplar optimization module that extends ProTeGi.
2. We formulate exemplar selection as a genetic search problem that jointly optimizes task performance, prompt length, and exemplar diversity.
3. We demonstrate consistent improvements over ProTeGi across multiple benchmarks and large language models, achieving higher average accuracy.¹

2 Background

2.1 In-Context Learning and Exemplar Selection

In-Context Learning (ICL) allows Large Language Models (LLMs) to solve new tasks using a small set of example inputs and outputs provided in the prompt, without changing the model itself [6]. The success of ICL depends on carefully choosing and arranging these examples. Studies show that the order of different examples can change the model’s accuracy by as much as 30% [7]. Including irrelevant examples in a

¹Code and data available at:<https://github.com/viet2005-68/APO.git>.

prompt can create ambiguity and add unnecessary complexity to the learning process. These examples may lead the model to focus on patterns unrelated to the intended task, increasing the risk of misinterpretation and limiting the model’s performance and generalization. Selecting concise, relevant examples is challenging but crucial to achieving robust in-context learning.

2.2 Automatic Prompt Optimization

To mitigate the manual effort in prompt engineering, Automatic Prompt Optimization (APO) techniques treat prompt design as a discrete optimization problem. A state-of-the-art framework is ProTeGi [4], which utilizes “textual gradients”, natural language critiques generated by the LLM, to guide the optimization of instructions. Specifically, the framework analyzes error cases to synthesize descriptions of defects and iteratively refines the instruction via beam search. However, one key shortcoming of existing techniques, such as ProTeGi, is that they are primarily concerned with improving instructions while ignoring the influence of optimizing the exemplar collection. These frameworks limit their ability to adapt to complicated tasks by failing to take advantage of the synergy between instructions and examples since they treat demonstrations as static components. [7]

3 Related Works

3.1 Prompts

Prompting has proved to be an effective use Large Language Models (LLMs) in accomplishing specific tasks. However, this relies heavily on how we write the prompt. In general, there are two basic types of prompting techniques: continuous and discrete prompt techniques.

Continuous approaches, also known as “soft prompts”, attempt to optimize the parameters of input tokens to control the model [8–10]. Even though these methods are efficient, they have two major flaws. First, they need to access to the internal parameters of the models and gradients, which is not always possible for black-box APIs. The second is that they are not easy to read for humans [11].

However, discrete ones are much simpler. All required is that we append natural language words or specific instructions, such as “Classify this comment”, to the input [12]. This approach is simpler to understand from a human perspective and has gave good results on many NLP tasks [13]. As a result of the aforementioned reasons, discrete tasks are more suitable for black box optimization.

3.2 Automatic Prompt Optimization Strategies

Various approaches have been suggested for the automation of prompt engineering. Early discrete methods used gradients or token probabilities to find optimal tokens [14, 15]. However, these methods often cannot be used for API-based models.

More recent efforts focus on gradient-free approaches. One line of work emphasizes. Though *exploration* by sampling many candidates to find the best prompt [3]. Another

line focuses on *exploitation* via adjusting prompts iteratively [16, 17] or working on errors to correct the instruction, as presented in the ProTeGi framework [4].

3.3 Exemplar Optimization (EO)

Most of the existing work on optimizing few-shot exemplars relies on **retrieval-based strategies** that find semantically similar examples [18, 19] or uses **model-generated demonstrations** [20]. Though recent toolkits like DSPy [21] embrace EO as a central paradigm, current research prevalently considers EO in isolation [22]. These methods often overlook how instructions and exemplars interact together. Furthermore, they typically test on simple tasks that do not pose a threat to state-of-the-art LLMs. As described in, we aim to bridge this gap by integrating evolutionary search with exemplars within the APO process.

4 The Proposed Method

The proposed method consists of two main phases: Instruction Optimization and Exemplar Optimization. We inherit the Instruction Optimization process from the ProTeGi [4], which iteratively refines natural language instructions using textual gradient feedback derived from model errors and selects high-performing candidates through beam search. This phase produces an optimized instruction p^* .

In the Exemplar Optimization phase, we assume access to the optimized instruction p^* obtained from the instruction optimization phase, along with training data consisting of input and output text: $D_{tr} = \{(x_1, y_1), \dots, (x_n, y_n)\}$. Note that p^* remains fixed throughout this phase, and the same D_{tr} is reused across both optimization phases. We interact with the large language model through a black-box API, which formulate as a function $LLM_p(\cdot)$. Given input x and the set of exemplars $E = \{e_1, \dots, e_k\}$ where $E \subseteq D_{tr}$, the prediction of the model is defined as:

$$LLM_P(x) \approx \arg \max_y P(y|p^*, x, E),$$

that returns a likely continuation of the text y of the concatenation of p^* , x and E . Our goal is to select the optimized set of exemplars $E^* \subseteq D_{tr}$ where

$$E^* = \arg \max_{E \subseteq D_{tr}} \mathcal{J}(E),$$

for the objective function $\mathcal{J}(E)$.

In the following sections, we first define the scoring function that jointly considers task performance, prompt length, and exemplar diversity, followed by a description of the genetic algorithm employed for exemplar optimization.

4.1 Objective Function

The goal of this optimization is to select a subset of exemplars $E \subseteq D_{tr}$ that combined with improved instruction prompt p^* maximizes downstream task performance, while controlling the length of prompt and reducing redundancy among examples.

Formally, we define the objective function as:

$$\mathcal{J}(E) = m(p^*, D_{tr}, E) - \lambda_{len}|E| - \lambda_{div}R_{div}(E)$$

where:

- $m(p^*, D_{tr}, E)$ denotes task specific metrics (e.g., accuracy) computed by comparing the LLM output on D_{tr} using instruction p^* and exemplar E
- $|E|$ denotes the length of exemplars set included to the prompt
- λ_{len} controls the strength of exemplars length regularization
- λ_{div} control the strength of diversity regularization

To encourage diversity among exemplars, and also to reduce overfitting to the training set, we define the diversity regularization term as the average pairwise similarity:

$$R_{div}(E) = \frac{1}{|E|(|E| - 1)} \sum_{i \neq j} sim(e_i, e_j)$$

where $sim(.,.)$ measures the semantic similarity between exemplars. We measure the semantic similarity through cosine similarity between sentence embeddings using model BAAI/bge-m3 [23] to embed exemplars.

This objective function assign a high score to the combination of optimized instruction p^* and exemplar set E when task specific metric is high, while simultaneously regularizing the size and diversity of exemplar set to reduce overfitting and promote generalization

4.2 Exemplar Optimization via Genetic Search

Optimizing $\mathcal{J}(E)$ is a combinatorial problem over the power set of D_{tr} , which is intractable to solve exhaustively. Moreover, the black-box nature of closed-source LLM prevents direct gradient-based optimization with respect to exemplar selection. To address these challenges, we employ a genetic algorithm (GA) [24] to efficiently explore the exemplar search space, as summarized in Algorithm 1.

Initialization

Each individual in the population represents a set of exemplars $E = \{e_1, \dots, e_k\}$, where k is constrained within a predefined range $k_{\min} \leq k \leq k_{\max}$ and population size is constrained with a predefined Q .

Error-Weighted Sampling.

Rather than using uniform sampling for initialization and mutation, we adopt a weighted sampling strategy that prioritizes misclassified examples, which are likely to capture harder or edge cases and provide more informative signals to the LLM.

We maintain an accumulated error count c_i for each training example $(x_i, y_i) \in D_{tr}$, reflecting how frequently the model fails to produce the correct output under the optimized instruction p^* . To bias exemplar selection toward informative examples, we

Algorithm 1 Exemplar Optimization via Genetic Search

Require: Optimized instruction p^* , training data D_{tr} , error counts $\{c_i\}$, optimize rounds t , population size Q , exemplar bounds $[k_{\min}, k_{\max}]$, temperature T , regularization coefficients $\lambda_{\text{len}}, \lambda_{\text{div}}$

Ensure: Optimized exemplar set E^*

- 1: Compute sampling probabilities: $P(e_i)$
 - 2: Initialize population $\mathcal{P}$ by sampling Q exemplar sets $E \subseteq D_{tr}$ with $|E| \in [k_{\min}, k_{\max}]$
 - 3: **for** $i = 1$ to t **do**
 - 4: **for** each $E \in \mathcal{P}$ **do**
 - 5: Construct prompt using (p^*, E)
 - 6: Evaluate fitness $\mathcal{J}(E)$
 - 7: **end for**
 - 8: Select top $Q/2$ exemplar sets as elites
 - 9: Generate offspring via mutation (add, remove, replace)
 - 10: Update population $\mathcal{P} \leftarrow \text{elites} \cup \text{offspring}$
 - 11: **end for**
 - 12: **return** $E^* = \arg \max_{E \in \mathcal{P}} \mathcal{J}(E)$
-

define a temperature-scaled sampling distribution:

$$P(e_i) = \frac{\exp((c_i - \max_j c_j)/T)}{\sum_j \exp((c_j - \max_j c_j)/T)},$$

where T is a temperature parameter controlling the degree of exploration. Initial exemplar sets are sampled without replacement according to this distribution.

Fitness Evaluation.

For each candidate exemplar set E , we construct a full prompt by inserting E into the exemplar section of the optimized instruction p^* . The resulting prompt is evaluated using the scoring function $\mathcal{J}(E)$, which balances task performance, exemplar set size, and exemplar diversity.

Selection and Mutation.

At each optimization round, individuals are ranked by their objective scores, and the top half of the population is retained as elites. New candidate exemplar sets are generated through mutation operations, including exemplar replacement, insertion, and removal, while respecting the constraints. Mutation candidates are sampled according to the same error-weighted distribution.

This process is repeated for a fixed number t optimization rounds, and the exemplar set with the highest objective score is selected as the final solution.

5 Experiments

Our experiments are designed to answer the following three research questions:

- **RQ1:** *Do exemplar optimization methods give better performance compared to purely instruction-only optimization frameworks like ProTeGi?*

In order to solve this question, a comparison has been made on two pivotal baselines:

(1) **Exemplar optimization using Random Search** represents a simple optimization algorithm. (2) **ProTeGi** [4] concentrates solely on the optimization of instruction. The purpose of the comparison is solely to assist in isolating the actual performance improvement achieved by adding an explicit exemplar selection strategy.

- **RQ2:** *Is the proposed genetic algorithm more effective and efficient than random search in exemplar search space?*

In this experiment, the goal is to extract the contribution of our search strategy alone. Our approach is compared fiercely against random selection baseline. This can ensure that performance improvements arise from the selection of diverse and relevant exemplars rather than stochastic variance.

- **RQ3:** *Does the proposed method show improvements across diverse types of classification benchmarks and different LLM architectures?*

To answer this question, several evaluations have been conducted by ours across **multiple diverse datasets** and **several different LLM architectures**. The goal of this thoroughly comparison is to confirm whether the proposed approach gives ours consistent as well as significant performance improvements regardless of the particular task or model.

We evaluate the effectiveness of the proposed method on six NLP classification tasks. Our approach is compared against ProTeGi as a baseline, and we observe consistent performance improvements on various tasks.

5.1 Datasets

We conduct experiments on six NLP classification benchmarks covering a broad range of reasoning and language understanding phenomena, including misinformation detection, hate speech classification, commonsense reasoning, and logical inference. All datasets are publicly available and have been widely used in prior work on prompt optimization and in-context learning. The evaluated datasets are:

- **LIAR** [25]: A benchmark dataset for fake news detection.
- **ETHOS** [26]: A textual dataset for online hate speech detection.
- **Causal Judgement** [27]: A BIG-bench Hard task evaluating commonsense causal reasoning.
- **Web of Lies** [27]: A BIG-bench Hard task focused on logical consistency and deception detection.
- **Sports Understanding** [27]: A BIG-bench Hard task testing factual and numerical reasoning in sports-related queries.
- **Boolean Expressions** [27]: A BIG-bench Hard task assessing logical reasoning over boolean expressions.

5.2 Setup

For each task, we select 100 examples for training set and 150 examples for testing set. All reported results are averaged of three different experiments. We use accuracy as the reported metric.

Both ProTeGi and our method Instruction Optimization stage ran on the same config. For the Exemplar Optimization stage, we set $Q = 12, k_{min} = 3, k_{max} = 6, t = 6, \lambda_{len} = 0.01, \lambda_{div} = 0.3, T = 1$

5.3 Results

Table 1 and Table 2 show the comparative performance of the proposed method, compared with the ProTeGi baseline and the Random Search strategy on six NLP classification tasks.

Performance on Qwen3-14B (Table 1).

Our method gives an average accuracy of 0.73 regarding the model Qwen3-14B, outperforming both the ProTeGi baseline with 0.71 and Random Search with 0.72. In particular, our approach illustrates clear effectiveness in tasks such as *LIAR* (+0.04) and *Boolean Expressions* (+0.06). While ProTeGi gains slightly better results on *Web of Lies* and *Sports Understanding*, the gap is slight. Importantly, the proposed evolutionary strategy exceeds the Random Search baseline in average accuracy, guaranteeing the improvement of the genetic algorithm in navigating the exemplar search space.

Performance on Qwen2.5-32B (Table 2).

It can be demonstrated from table, the effectiveness of our approach is further amplified on the larger Qwen2.5-32B-Instruct-AWQ model. As illustrated in Table 2, our method acquires the highest average accuracy of 0.79, clearly outperforming ProTeGi (0.75) and Random Search (0.77). It is notable that, in the *Boolean Expressions* task, our method achieves a substantial gain, seeing an increase in accuracy from 0.84 to 0.91. Consistent increments are also observed in *Web of Lies* (+0.06) and *Sports Understanding* (+0.05).

Overall Analysis.

To sum up, the proposed *ProTeGi + EO (Ours)* achieves the highest average results. These indicate that while random exemplar selection can provide some merits over fixed instructions (0.72 vs 0.71 in Table 1; 0.77 vs 0.75 in Table 2), the guided evolutionary search gives a superior and more robust optimization. It can be confirmed that optimizing instructions and exemplars results in better generalization than instruction tuning alone and scaling effectively to larger models.

6 Conclusion

The goal of this research is to enhance prompt optimization by introducing a novel technique built on an existing framework available called ProTeGi. Rather than

Table 1: Model: Qwen3-14B

Task	ProTeGi	ProTeGi + EO (Random Search)	ProTeGi + EO (Ours)
LIAR	0.57	0.59	0.61
ETHOS	0.86	0.89	0.89
Causal Judgement	0.63	0.65	0.68
Web of Lies	0.54	0.52	0.53
Sports Understanding	0.82	0.82	0.80
Boolean Expressions	0.82	0.87	0.88
Average	0.71	0.72	0.73

Table 2: Model: Qwen2.5-32B-Instruct-AWQ

Task	ProTeGi	ProTeGi + EO (Random Search)	ProTeGi + EO (Ours)
LIAR	0.65	0.67	0.69
ETHOS	0.87	0.89	0.90
Causal Judgement	0.70	0.65	0.69
Web of Lies	0.66	0.68	0.72
Sports Understanding	0.79	0.81	0.84
Boolean Expressions	0.84	0.9	0.91
Average	0.75	0.77	0.79

adhering to traditional processes, there is an extra phase involving the careful selection of examples to strike an optimal balance between the length and coverage of prompts. The new solution was applied to six different natural language processing (NLP) tasks and achieved superior performance compared with standard methods relying on instructions. The results indicate that intentionally choosing examples, alongside refining instructions, can greatly improve prompt effectiveness without significant additional expense.

References

- [1] OpenAI: GPT-4 Technical Report (2024). <https://arxiv.org/abs/2303.08774>
- [2] Nie, A., Cheng, C.-A., Kolobov, A., Swaminathan, A.: The Importance of Directional Feedback for LLM-based Optimizers (2024). <https://arxiv.org/abs/2405.16434>
- [3] Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J.: Large Language Models Are Human-Level Prompt Engineers (2023). <https://arxiv.org/abs/2211.01910>
- [4] Pryzant, R., Iter, D., Li, J., Lee, Y.T., Zhu, C., Zeng, M.: Automatic Prompt Optimization with "Gradient Descent" and Beam Search (2023). <https://arxiv.org/abs/2305.10542>

[org/abs/2305.03495](https://arxiv.org/abs/2305.03495)

- [5] Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., Yang, Y.: EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers (2025). <https://arxiv.org/abs/2309.08532>
- [6] Brown, T.B., Benjamin Mann, e.a.: Language Models are Few-Shot Learners (2020). <https://arxiv.org/abs/2005.14165>
- [7] Wan, X., Sun, R., Nakhost, H., Arik, S.O.: Teach Better or Show Smarter? On Instructions and Exemplars in Automatic Prompt Optimization (2024). <https://arxiv.org/abs/2406.15708>
- [8] Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: ACL (2021). <https://arxiv.org/abs/2101.00190>
- [9] Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., Tang, J.: Gpt understands, too. In: AI Open (2021). P-Tuning. <https://arxiv.org/abs/2103.10385>
- [10] Zhang, N., Li, L., Chen, X., Deng, S., Bi, Z., Tan, C., Huang, F., Chen, H.: Differentiable prompt makes pre-trained language models better few-shot learners. arXiv preprint arXiv:2108.13161 (2021)
- [11] Lester, B., Al-Rfou, R., Constant, N.: The power of scale for parameter-efficient prompt tuning. In: EMNLP (2021). <https://arxiv.org/abs/2104.08691>
- [12] Schick, T., Schütze, H.: Exploiting cloze questions for few shot text classification and natural language inference. In: EACL (2021). <https://arxiv.org/abs/2001.07676>
- [13] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. ACM Computing Surveys (2023)
- [14] Shin, T., Razeghi, Y., Logan IV, R.L., Wallace, E., Singh, S.: Autoprompt: Eliciting knowledge from language models with automatically generated prompts. In: EMNLP (2020). <https://arxiv.org/abs/2010.15980>
- [15] Shi, W., Michael, J., Hannan, S., Zettlemoyer, L.: Toward human-readable prompt tuning. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (2022)
- [16] Zhang, T., Wang, X., Zhou, D., Schuurmans, D., Gonzalez, J.E.: Tempera: Test-time prompting via reinforcement learning. In: ICLR (2023)
- [17] Prasad, A., Stengel-Eskin, E., Bansal, M.: Grips: Gradient-free, edit-based instruction search for prompting llms. In: Proceedings of the 17th Conference of

- the European Chapter of the Association for Computational Linguistics (2023). Preprint arXiv:2206.02239 (2022)
- [18] Lu, Y., Bartolo, M., Moore, A., Riedel, S., Stenetorp, P.: Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In: ACL (2022). <https://arxiv.org/abs/2104.08786>
 - [19] Wu, Z., Wang, Y., Ye, J., Kong, L.: Self-adaptive in-context learning. In: ACL (2023). <https://arxiv.org/abs/2212.10375>
 - [20] Kim, H.J., Cho, H., Kang, J., Kim, T., Lee, S.-g., Kim, T.: Self-generated in-context learning: Improving limited-resource nlu with synthetic examples. In: NAACL (2022). <https://arxiv.org/abs/2206.08082>
 - [21] Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., *et al.*: Dspy: Compiling declarative language model calls into self-improving pipelines. In: The Twelfth International Conference on Learning Representations (2024). <https://arxiv.org/abs/2310.03714>
 - [22] Margatina, K., Vernikos, G., Barranguet, L., Aletras, N.: Active in-context learning: An information-theoretic approach to example selection. In: EMNLP (2023). <https://arxiv.org/abs/2302.12246>
 - [23] Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation (2025). <https://arxiv.org/abs/2402.03216>
 - [24] Goldberg, D.E.: Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, Reading, MA (1989)
 - [25] Wang, W.Y.: "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection (2017). <https://arxiv.org/abs/1705.00648>
 - [26] Mollas, I., Chrysopoulou, Z., Karlos, S., Tsoumakas, G.: Ethos: a multi-label hate speech detection dataset. *Complex and Intelligent Systems* **8**(6), 4663–4678 (2022) <https://doi.org/10.1007/s40747-021-00608-2>
 - [27] Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H.W., Chowdhery, A., Le, Q.V., Chi, E.H., Zhou, D., Wei, J.: Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them (2022). <https://arxiv.org/abs/2210.09261>